UNIT 13 IMPLEMENTATION STRATEGIES - 2

Structure **Page No.**

13.0 Introduction

13.1 Objective

13.2 Creating Methods from Collaboration Diagram

13.3 Implementing Constraints

13.4 Implementing State Charts

13.5 Persistency

13.6 Summary

13.7 Solutions/Answers to Check Your Progress

13.8 References/Further Readings

13.0 INTRODUCTION

As you know, object-oriented software development methodology uses three different facets: Object model, Dynamic model and Functional model. The object model defines objects and their relationships in the system; dynamic model describes control structure of objects, and the functional model describes the data transformations of the system. The previous unit of this block introduced you to the conversion of design diagram to executable code, where you learnt the implementation of object model concepts like class and their objects and different associations. This unit is also on the line and gives implementation strategy for the dynamic behavior of object-oriented system. In the next unit, which is the final and last unit of this course, you will learn how to transform object Classes into Tables.

The collaboration diagram is an UML diagram that comes under the Interaction diagram and depicts the relationships between the objects in a system. It defines a functionality of the system by demonstrating a set of chronological interactions between objects.This unit will introduce you to how to create methods from collaboration diagram. In addition to this, you can also learn about the implementation of constraints as well as statecharts. The Constraints represent semantics of object-oriented model elements such as objects, links and attributes. The statechart diagram is used to designate the behavior of a particular class. This unit will cover different statechart implementation strategies such as switch statement, helper object, and collaborator object. At the end of this unit, you will know about the persistency concept.

13.1 OBJECTIVES

After going through this unit you shouldl be able to explain:

- how to create method from collaboration diagram,
- constraints and their implementation,
- statecharts and their implementation, and

- difference between persistence and non-persistence data.

13.2 CREATING METHODS FROM COLLABORATION DIAGRAM

The collaboration diagram is an UML diagram which depicts the relationships between the objects in a system. It defines a functionality of the system by demonstrating a set of chronological interactions between objects. It is basically used to demonstrate how objects interrelate to perform the behaviour of a specific use case. A collaboration diagram carries the similar type of information as a sequence diagram; it focuses on the object structure instead of the chronology of messages passing between them. The sequence and the collaboration diagrams come under the same head as the interaction diagram. The collaboration diagram is also called a communication diagram and is used to represent the object's architecture in the system.

The key purpose of creating this diagram is to illustrate the relationship between objects and works together to perform a particular task. This task is difficult to regulate from a sequence diagram. Besides, collaboration diagrams can also support you in controlling the accuracy of your static model, such as class diagrams.

In an application system, there are one or more collaboration diagrams to find all the eventuality of a complex behaviour. While designing the collaboration diagram, it contains the elements such as actors, objects and their communication links, and messages. Each actor performs a significant role as it invokes the interaction and has a name. The link behaves as a connector between the objects and actors. It depicts a relationship between the objects through which the messages are sent. The messages are exchanged between objects in an order which is represented by sequence numbers. The messages are sent from the sender to the recipient, and the direction must be traversable in that particular direction. Collaboration diagrams are appropriate for analyzing use case diagrams. This diagram can be used to show the dynamic behaviour of a specific use case and describes the role of each object.

You can create collaboration diagrams using StarUML, ArgoUML and many others UML designing software tools. You can also generate collaboration diagram automatically from the sequence diagram. While designing collaboration diagrams, you can follow the basic steps:

- First, you can determine the scope of the collaboration/communication diagram. The scope of a collaboration diagram can be a use case.
- You can put the objects which play a part in the collaboration on the diagram. While putting objects, the most important objects are to be placed in the centre of the diagram.
- If a specific object has a property or retains a state that is important to the collaboration, you can set the initial value of the property or state.
- All objects should be associated with links, and each link should be associated with message(s) or method(s) calls.
- You can add sequence numbers to each message corresponding to the time-ordering of messages in the collaboration.
- You can represent ‘if else’ condition in a collaboration diagram asshown in figure 13.1.

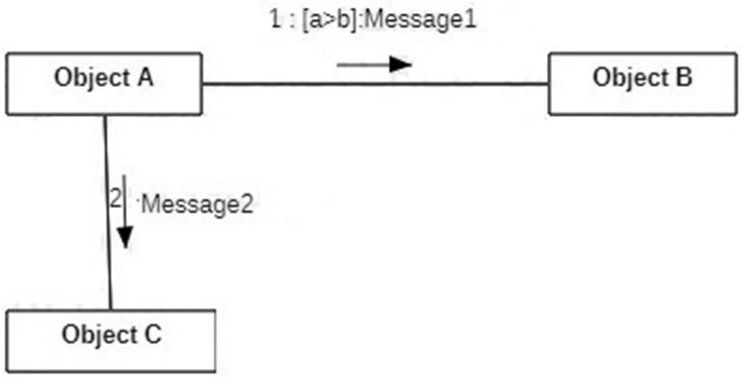

Figure 13.1: Collaboration Diagram showing If-else condition

The implementation of the above diagram is as follows:

```
if (a>b)
{
    ObjectB.Message1();
}
 else
 {
   ObjectC.Message2();
}
```

Let us consider a collaboration diagram for ATM transactions. For this example, first, you create a Use Case diagram and then draw a collaboration diagram. For building a Use Case diagram, you can use the below flow of controls:

- Customer can insert ATM card into the card reader of the ATM console.
- The card reader reads and verifies the card.
- ATM console displays a message and prompts the customer to enter PIN.
- Customer enters PIN.
- ATM console verifies PIN with bank database and confirms the PIN is valid.
- If PIN is **invalid**, ATM console notifies the customer with a message and ejects the card.
- If PIN is **valid,** ATM console represents a prompt with options such as deposit, withdraw and transfer money.
- Customer selects withdraw option.
- ATM console presents prompt to the customer for an accurate amount to be withdrawn.
- The customer enters the amount.
- ATM console verifies with the bank database whether the customer account has sufficient funds. Otherwise, it shows an insufficient funds message.
- ATM console deducts money from customer's account, puts the requested amount in cash dispenser, and prints a receipt.
- ATM console rejects the card and becomes ready for another transaction.

Using the above flow of messages, you can construct the collaboration diagram similar to figure 13.2 where messages/method calls are flowing between objects with

a sequence number. This sequence number specifies the ordering of the messages. The messages appear with pointing arrows from the sender object to the receiver object.

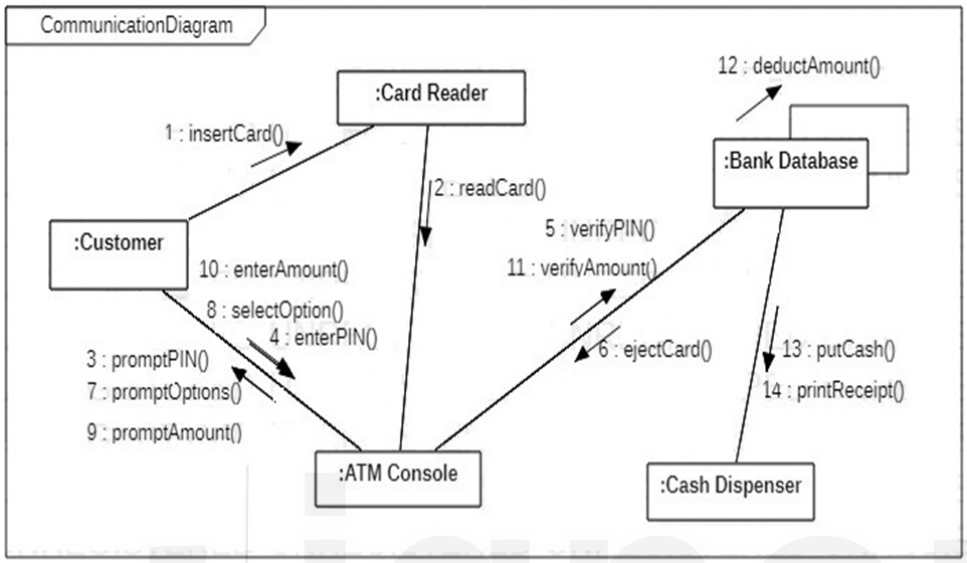

Figure 13.2: Collaboration Diagram for ATM transaction

When you create a collaboration diagram for the real business application , you can create methods as per need. The classes and their methods are created in the Collaboration Diagram given in figure 13.2, which can be summarized in the tabular form:

Table 13.1: Description of Classes and Methods

Class Name	Methods	Class Name	Methods
Customer	insertCard() enterAmount() selectOption() enterPIN()	Card Reader	readCard()
ATM Console	promptPIN() verifyPIN() promptOptions() promptAmount() verifyAmount()	Bank Database	ejectCard() deductAmount() putCash() printReceipt()

Now you can transform methods defined in the collaboration diagram into executable code. You inscribe source code yourself or use software tools such as Eclipse and Concurrent Versions System (CVS). There are number of methods in the figure-13.2, but the pseudocode given below is written for verifyPIN() method only. This method verifies the customer's Personal Identification Number (PIN) using data retrieved from the ATM card's magnetic strip. This method asks the customers to enter their PIN using the keypad of ATM console. If the PIN does not equal to the PIN stored on the card, then the ATM system allows a limited number of repeats. If it is still not matched, then the system invokes ejectCard() method and the card is seized as a

security precaution. If the customer enters the correct PIN then the system invokes promptOptions() method where it provides options such as deposit money, withdraw cash and transfer money. You may try to write code for other methods as well.

```
method verifyPIN
constants no_of_maxpins is 2
variable pin, pin_count is number
set pin_count to zero
read pin from ATMcard
loop until pin_count is equal to no_of_maxpins
input pin from customer
if entered pin matches with cardpin_database
then call promptOptions method
endif
add 1 to pin_count
endloop
if pin_count is equal to no_of_maxpins
then seize customer's card
else call ejectCard method
endif
endmethod// verifyPIN
```

Consider another example of 'Programme Management System'. Using this example, you can learn about the creation of methods from the collaboration diagram. This system contains classes like Admin, ProgrammeCoordinator, Programme, Course and Faculty. Admin initiates the manage programme functionality and interacts with Programme Coordinator. The ProgrammeCoordinator intermingled with different object classes in the system, such as Programme, Course and Faculty. The Programme Coordinator starts the process of creation and modification functionality of the programme. After the programme is either created or modified, ProgrammCoordinator is responsible for the creation or modification functionality of a course. Each programme consists of many courses. It means that the class Programme and class Course are associated with each other in the form of one-to-many relationships. The ProgrammeCoordinator assigns the course(s) to faculty. Finally, the admin invokes the assigned faculty to the Course functionality, of the Programme. The Collaboration Diagram for Programme Management System is shown in figure 13.3:

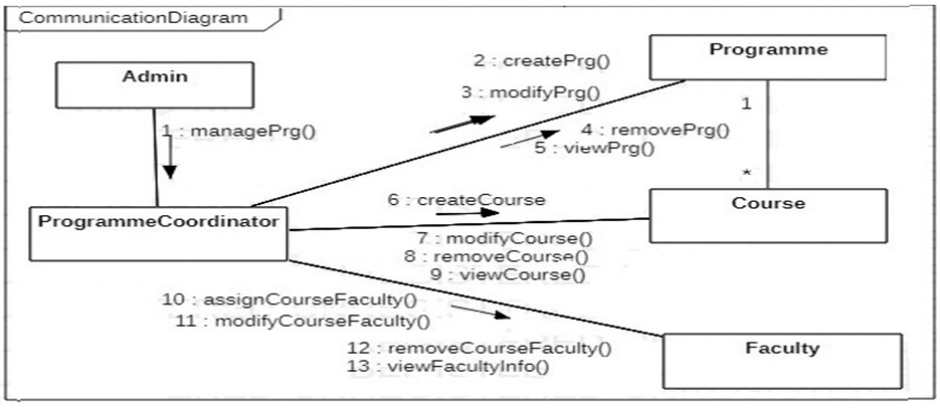

Figure 13.3: Collaboration Diagram for Programme Management System

The collaboration diagram for 'Programme Management System' is shown in figure 13.3. However, the classes are involved in this collaboration diagram for 'programme management system, which contains the following methods. You may write code for the methods given in figure 13.3. List the basic assumptions, then try to write the code in the programming language you are comfortable.

Table 13.2: Description of Classes and Methods in Collaboration Diagram

Class Name	Methods	Class Name	Methods
Admin	managePrg()	Faculty	assignCourseFaculty() modifyCourseFaculty() removeCourseFaculty() viewFacultyInfo()
Programme	createPrg() modifyPrg() removePrg() viewPrg()	Course	createCourse() modifyCourse() removeCourse() viewCourse()

☞ **Check Your Progress - 1**

1. What is Collaboration Diagram? Why do we need collaboration diagrams?

   ---

2. Explain the basic elements that should be considered for defining collaboration diagrams?

   ---

3. What is the difference between collaboration and sequence diagram?

13.3 IMPLEMENTING CONSTRAINTS

As you know, if any product is well designed, it will yield fruitful results. In the similar manner, if you want to design a software application with the sound quality of information, then it should be designed with appropriate constraints in the analysis and design phase. If your application is not well designed, it will be erroneous and more expensive to correct. In general terms, constraint means restriction or limitation that is used for optimization purposes. This section describes how to implement constraints in terms of object-oriented paradigms.

A constraint is a condition that restrains the semantics of an element and it must always be true. For example, a person can only get a loan from a bank if the person has an account with the bank. A constraint is either to be declared in the tool and used in several diagrams or identified and applied as needed in a diagram. Some constraints are predefined in UML diagram, and others may be user-defined. When you build a business application, they are closely associated with the constraints. Using the constraints, you can control the software behaviour. You can say that constraint is a rule used for optimisation.

Constraints are used as semantic checks that are implicitly satisfied by a given set of objects. For example, constraints can control the range of an object's integer attribute. You can say that the constraints are defined as rules to be applied to classes and/or fields (attributes) of the class. For example, if a rule dictates that the employee name may never be null, then you would like to place the @NotNull annotation to define the constraint like the following way:

```
public class Employee
{

   @NotNull
   private String empName;

   // ...
}
```

The constraints are used to represent the semantics of object-oriented model elements such as objects, links and attributes. You can use constraints to reflect the rules of the problem domain in the analysis model. Constraints define properties which must be true at run time for the entire system. Usually, constraints do not have names that recognise the contents of their bodies. A constraint is written as a text and appears in a rectangle enclosed with braces with a folded upper-right corner.

You can define constraints in each of the three models of the object-oriented approach. It defines the relationship between the objects of an object model. It also describes the relationship between states and the events of different objects in dynamic modeling, whereas in functional modeling, constraints are used to apply restrictions on the transformations and computations. Constraint contains conditions or restrictions which are included by writing code in an application and verify the current state of the application at run-time. When these constraints are included in the class, it automatically applied to its instances.

You can use a constraint on a single element, such as a class or an association path; the text for constraints may be positioned near the name of the class element and placed in a **note** symbol. A constraint is written as a text string in curly braces using the following syntax:

```
constraint ::= '{' [  name ':' ] boolean-expression '}'
```

Constraints on Objects

The following class diagram demonstrates an example of applying constraints on the attributes. Employee could be as a boss or the worker. Worker's salary cannot exceed the salary of boss. You can see figure 13.4 for the syntax of constraints. It is defined as condition within the braces, placed near the constrained object and then connected with dotted line.

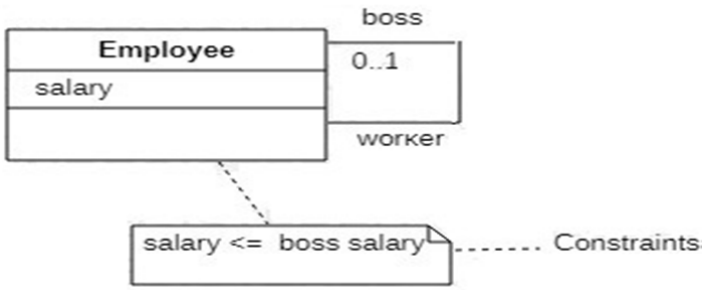

Figure 13.4: Constraints on objects

Constraints on Links

As you know, the multiplicity specifies how many objects of a class can be coupled with an object of the other association class. The multiplicity constraints restrict the number of objects associated with the given object. Object modeling notations define the syntax for displaying multiplicity values like [0, 1] and exactly 1. It has a special notation "{ordered}" which designates the elements of "many" side of an association have an explicit order that must be maintained. The following object model, as shown in figure-13.5, illustrates the Regional Centres of University. Each Regional Centre for each university has a set of employees who have held the regional centre, ordered chronologically.

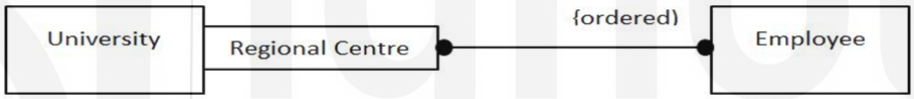

Figure 13.5: Constraints on Links

Constraints are commonly used for various elements of class diagrams. They work as a bridge to develop functional relations between the entities of an object model. Constraints are helpful to communicate the semantics of classes, validate the correctness of object states at run-time, and repair inconsistent data structures and for various types of reasoning.

The following example of a class diagram represents two classes, Candidate and Job, as shown in figure 13.6. The constraints condition is written near the class which implies on both the classes. The candidates do not apply for the job if the age of the candidate is more than 30.

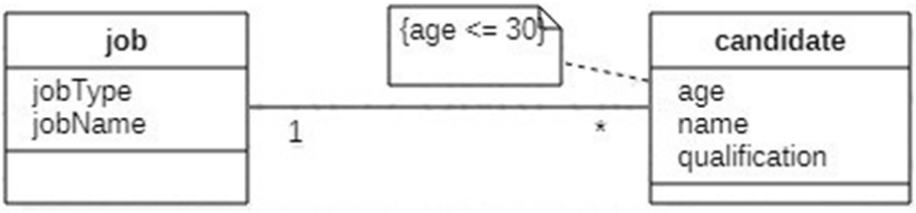

Figure 13.6: Constraints Example

As yet, you know that the constraint is a condition. The use case works under the pre-, post- and invariant conditions. A precondition states the condition that is required to be met before the use case can proceed. A post-condition specifies the condition which must be true after the execution of the use case. An invariant condition is that condition which is true during the execution of the use case.

Let us consider another class diagram and implementation for constraints, as shown in figure 13.7.

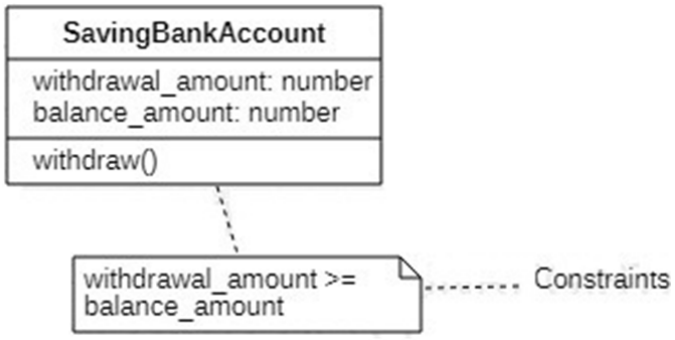

Figure 13.7: Constraints on objects example

The implementation of the above class diagram is as follows:

```
public class SavingBankAccount
{
public void withdraw(double withdrawal_amount)
{
private balance_amount;
          if (withdrawal_amount >=  balance_amount)
          {   …..
           balance_amount = balance_amount–withdrawal_amount;
            }
          //show error message
}
}
```

There is a number of constraints such as not null, unique, primary key, and foreign key in RDBMS, which you will study in the next unit of this block. A NOT NULL constraint defines a rule which protects a column to not accept null values. The UNIQUE constraint makes sure that all values in a column are different. A PRIMARYKEY constraint is used to identify each row in a table uniquely. A FOREIGN KEY is a primary key of one table that is embedded in another table. You can use primary and foreign key constraints to define the relationship between tables.

When these constraints are added to a relational table, then the table to be tested to confirm that its contents do not oppose the constraint. If the table contents infringe the constraint, the database server returns an error and does not add such constraint.

13.4 IMPLEMENTING STATE CHARTS

In the previous section, you have learnt about the implementation of the constraints. This section describes how to convert statecharts into executable code in object-oriented language like java.

You are already aware that the object model defines objects, attributes and links between them. The attribute values and links detained by objects are known as its **state**. When one object communicates with another, the communication result of these objects is in the form of **events**. Events are external or internal features influencing the system. The state chart or diagram represents the combination of states, events and transition (relationship between two states). This diagram is used in dynamic modeling to define a class's dynamic behaviour. The dynamic model contains multiple statecharts and one for each class.

The following state diagram for booking tickets is shown in figure 13.8 :

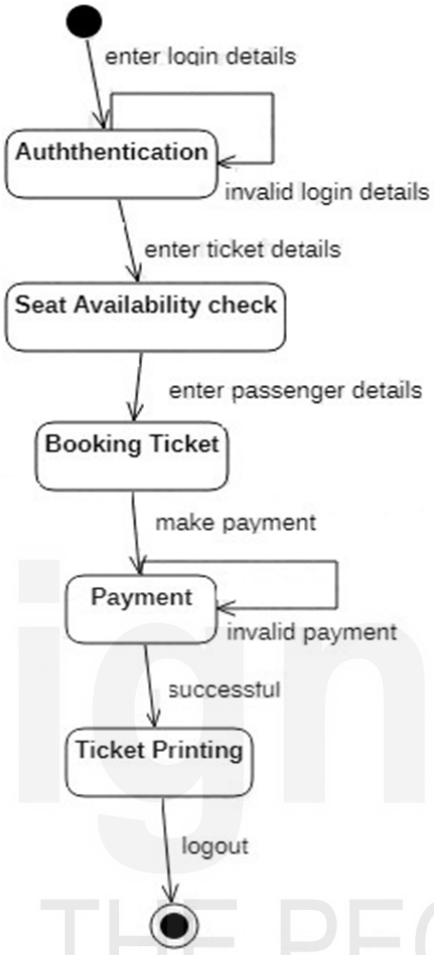

Figure 13.8: State diagram for Booking Tickets

The UML statechart diagram is used to designate the behaviour of a particular class. This diagram can be included only at the class level. A class can have either a statechart or an activity diagram but cannot have both together. While defining the states, you can ignore those attributes that do not affect the object's behaviour. Some classes do not practice statecharts since their objects are uni-state, and they always behave in the same manner.

The simple format of the state diagram is shown in the following figure 13.9:

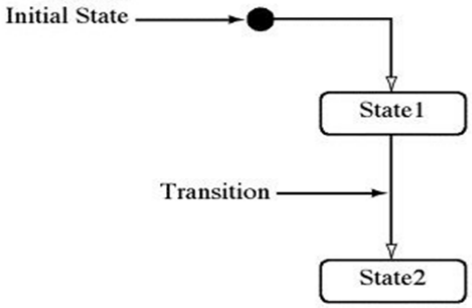

Figure 13.9: Simple Format for State Diagram

There are some strategies for implementation of a statechart diagram such as switch statement, helper object as well as collaboration object.

Implementing Statechart using Switch Statement

This section describes the switch statement approach for implementing statecharts. The states of statecharts diagrams are denoted as objects or scalar variables. Events and actions are indicated as methods.

The most basic technique to implement statechart is the switch statement. You may be aware of the functionality of a switch statement in java where a switch statement executes one statement from multiple case statements depending upon the condition. It is based on the current active state and control transfer to the code for processing the event. It is just like an if-else-if ladder statement. Each case clause of the switch statement denotes a single state of the object and can implement the various actions and activities for the specific state. The actions are implemented as a simple statement. The behaviour of the statechart is placed in a single class. States are denoted as data values. A single scalar variable is a state variable that stores the current active state. The state variable is used inside each event method of the context class. The right case is chosen on the value of the state variable.

This implementation strategy is simple, but it has drawbacks also. In the case of the addition of a new state in statechart, this approach does not allow flexibility in the code. There is code duplicacy in many places, and the reuse of code is very difficult.

Implementing Statechart using Helper Object

Due to the redundant code problem in the switch statement, the notion of a helper object is introduced. This is an alternative object-oriented approach to the switch statement. It places each case clause of a switch statement in a separate object. It provides a natural way to implementation of the statechart diagram. It removes the code redundancy and builds reusable code.

The helper object denotes the current state of the domain object and specifically implements the behaviour of the current state. The domain object transfers all external messages to its helper object, which reacts to the messages on account of the domain object. The helper object signifies only one state of the domain object. The methods of helper objects do not enclose conditional statements and are very simple ways to implement. Whenever changes occur in the state of the domain object, a new helper object is created, which interchanges the older current state and implements the behaviour defined in the new state. The reference to the helper object is recognized as the current state in the domain class.

The statechart shows the behavior of the class, which is a super context or domain class. During this implementation, an abstract state class is created, which describes the interface for capturing the behaviour connected with the states of the statechart. Each state turns into a derived class from the abstract state class. Each transition transforms into a method in the corresponding state class to provide an appropriate way of invoking services on the context object. Each action in statechart will be a method in the context class which contains the reference of the current state in the state object and offers all events for processing to the current state object.

Consider the following statechart diagram shown in figure 13.10 for the video player application with two states: Stop and Play. In the Stop state, when a user presses a play button, then *playButton* event happens, and the device executes the *Playstart* action and goes to ‘Play’ state. In the Play state, when the user presses a stop button, the stopButton event generates, and the device executes the *Playstop* action and goes to the Stop state.

While implementing the statechart as shown in figure 13.10, there are four classes: ' VideoPlayer’, ‘PlayerState’, ‘Stop’ and ‘Play’. The VideoPlayer is super context class and PlayerState is an abstract state class. The two classes such as Stop and Play which

implement the behavior specific to the Stop and Play states respectively and they are subclasses of PlayerState class which offers a common interface for all state classes. These classes are called state classes since they implement individual states of the statechart involved in the domain class.

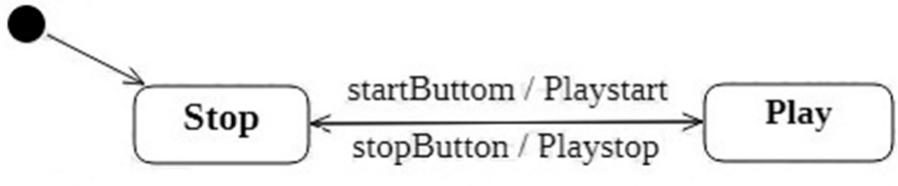

Figure 13.10: Statechart for a video player

These state classes have a reference to the domain object and name it as 'vPlayer' (as mentioned in the program). In brief, each state in the statechart diagram turns into a class, and each transition from that state becomes a method in the corresponding class, and all actions become methods in the domain class. The domain object forwards the all incoming messages to its current state object, which is mentioned in the program code under the name of *ps*.

The behaviour of the above state diagram can be implemented in Java as follows:

```
class VideoPlayer
{
        PlayerState ps; // helper object
        VideoPlayer()
                {
                ps = new Stop(this);
                }
        void stopButton()
                {
                ps.stopButton();
                }
        void playButton()
                {
                ps.playButton();
                }
        void Playstart() {...}
        void Playstop() {...}
}
//abstract state class
class PlayerState
{
        VideoPlayer vPlayer;   //reference to domain object
        PlayerState(VideoPlayer vp) {//constructor
        vPlayer = vp; }
        void stopButton() {}
        void playButton() {}
}
//Stop subclass
class Stop extends PlayerState
{
        Stop(VideoPlayer vp)
                {
                super(vp);
                }
        void playButton()
         {
        vPlayer.Playstart();
        vPlayer.ps = new Play(vPlayer);
```

```
        }
}
//Play subclass
class Play extends PlayerState
 {
        Play(VideoPlayer vp)
        {
        super(vp);
        }
        void stopButton()
        {
        vPlayer.Playstop();
         vPlayer.ps = new Stop(vPlayer);
        }
}
```

Implementing Statechart using Collaborator Object

Even though helper object provides enhanced remedy than the switch statement for the implementation of statechart diagram, we always believe that there is scope for better improvement in any technique. This section describes the collaborator object approach for implementing statechart diagram.

In the collaboration object approach for implementing statechart, the context class or domain class behaviour is fragmented into context and a state. The context object is an instance of the main class. Using this context object, the client communicates with a collaborator object, which symbolises the behaviour in one of its states.

The collaborator object contains all the state precise actions of the context object. The context object retains a collaborator object which represents to an instance of the current active state object and maintains references of all the state objects. They are created once in the constructor of the context class, and this class is accountable for setting the new state by changing the state reference in the collaborator object. The states are denoted as objects which implement state precise behaviour. The events are signified as methods in the context class. The context object delegates all events to the collaborator object for processing. Unlike helper objects, no new object is created for State Transitions.

An abstract state class is defined, which maintains a reference to access the context class. The statechart diagram contains one or more states, and each state grows as a class which is derived from the abstract state class. You may assign a similar name to the state and the class. The state classes implement the state-specific behaviour. The state object contains attributes and behaviour for a particular state. In this approach, the context object defines the transitions. Each state transition turns into a method in the corresponding state class, providing a way to invoke services on the context object. All the state-specific code resides in one class.

This way, you can implement a statechart in various modes. More you practice more you learn.

☞ Check Your Progress - 2

1. What is a constraint? How do you show constraints on a class diagram?

   ---

2. What is statechart diagram? What is the purpose of this diagram? How to implement a statechart into an executable code?

13.5 PERSISTENCY

In general meaning, persistent means preservation. It occurs over a prolonged period. This section describes the persistency in terms of object-oriented approach.
An object resides in a memory space and survives for a specific period of time. In traditional programming, the life of an object is usually the life of the execution of the program that created it. In files or databases, the object life is longer than the duration of the process creating and using the object in a program. This property by which an object stays even after its initiator concludes (close the program) is known as **persistence**. Persistence is important for enterprise applications because it gives required access to relational databases.

Data Persistence

You all know that computer programs operate on data. The most important tasks the application performs are to save and restore data. Persistent data may give useful mechanism for passing data between programs and permits the program to resume processing after a while. If you want data to persists after the program execution, then you must use a permanent data store.

Process Persistence

Persistence represents the lifetime of a process or an object. Referring to the computer threads and processes, a persistent process is a process which cannot be killed or shut down by other processes until the user kills them.

Object Persistence

In connection with data, persistence means an object should not be removed except it is actually erased. This requires appropriate storage and certain procedures which permit the data to persist. You can store persistent data in a persistent database, appearing as long-lasting records or long-lasting objects that fluctuate within devices and software. **Persistent data** is steady and restorable and is accessible after closing the application. This type of data must be saved into a database or internal or external memory, whereas **Non- persistence data** is not accessible after closing the application. The non - persistence data means it is volatile data that is available during the application's execution. For example, relational database management systems store persistent data in the form of records and tables and are accessible even after closing the application.

Identifying Persistent Data

While designing models, models may not always provide a clear picture of the data, and this data should be persistent or transient. There are no limitations to defining permanent data in models, so a single model can bring together persistent and transient data. The **transient data** are temporary in nature and do not have any representation and identifier value in the databases. Transient data is stored in random access memory and valid only inside a program; it is lost whenever the program terminates. **Persistent data** are saved to the database and it survives after transaction updates or program concludes. Generally, persistent data is used to specify the shared, accessed and updated databases across transactions.

Serialization

Serialization is the process of transforming an object into byte streams to store the object to memory or a database. The serialisation's main objective is to save an object's state so that it recreates when needed. The inverse process is known as **deserialization,** and this process is used to convert byte streams to actual Java objects in memory. The creation of the byte stream is platform-independent. So, the object serialized on one platform can be deserialized on a different platform.

Java provides serialization with the use of 'Serializable' interface. This interface (java.io.Serializable) is a marker interface which does not declare any data members and methods. Using this interface, "mark" the java classes, and objects of these classes may get a certain ability. When it is done, the methods 'writeObject' of ObjectOutputStream classes used for serializing an Object and 'readObject' of ObjectInputStream class is used for deserializing an object. This way, persistence can be implemented in java.

The next unit will explain to you how to map Objects to tables in the database.

☞ Check Your Progress - 3

1. What is the difference between persistent and transient objects?

   ---

2. What do you understand by Data Persistence? How you will make your data persistent?

   ---

3. What is the difference between persistent data and non-persistent data?

   ---

4. What is serialization? Where it is used, and why?

13.6 SUMMARY

This unit discussed the collaboration diagram; implementing constraints and statechart in object oriented paradigms. The key purpose of creating collaboration diagram is to illustrating the relationship between objects and to explain how objects work together

to perform a particular task. It is also called a communication diagram. This unit has also covered about the constraints, which is useful for controlling the software behaviour. A Constraint is a rule that is used for optimization purpose.

Futher you have been t explained t the implementation of behavioral features of model through the statechart. The dynamic model contains multiple state charts and one for each class. Some of the statechart implementation techniques are also discussed, such as switch statement, helper object as well as collaborator object. The switch statement is the most basic and earliest technique to implement a statechart. Due to the redundant code problem in switch statement, an alternative object-oriented approach as helper object approach is introduced for implementing statecharts. One more improved approach as collaborator object has also been introduced to implement statechart diagrams.

Finally, persistency concepts have been discussed concerning object oriented databases and relational databases. The Persistence data is accessible after closing the application. This data must be saved into a database or internal or external memory. Non- persistence data is not available after closing the application. The non - persistence data means it is volatile data that is available during the application's execution. You have also known about serialization, which is transforming an object into byte streams to store the object in memory, a database or a file.

13.7 SOLUTIONS/ANSWERS TO CHECK YOUR PROGRESS

☞ Check Your Progress - 1

1. The collaboration diagram is a UML diagram which depicts the relationship between the objects in a system. It defines a functionality of the system by demonstrating a set of chronological interactions between objects. It is used to demonstrate how objects interact to perform the behaviour of a use case. The purpose of collaboration diagrams is to visualize the interactive behaviour of the objects in a system. It is used to capture the dynamic behaviour of the system.
2. Four basic elements are used to construct the collaboration diagram: objects, actors, links and messages. Each actor performs a significant role as it invokes the interaction and has a name. The link behaves as a connector between the objects and actors. It depicts a relationship between the objects through which the messages are sent. The messages are exchanged between objects and include sequence numbers.
3. The difference between collaboration and sequence diagrams are as follows:
   - The collaboration diagram is used to depict the relationship between the objects in a system. The sequence diagram is used to visualize the sequence of calls in a system which is used to implement a precise functionality.
   - The collaboration diagram is used to denote the system's structural architecture and the messages sent and received. The sequence diagram is used to signify the sequence of messages that are transmitted from one object to another.
   - The collaboration diagram focuses on the object's interaction, whereas the sequence diagram concentrates on the time sequence.

☞ Check Your Progress - 2

1. A constraint is a condition that restrains the semantics of an element, and it must always be true. For example, a person can only get a loan from a bank if the person has an account with the bank. Some constraints are predefined in

UML diagram, and others may be user-defined. The constraints are used to represent the semantics of object-oriented model elements such as objects, links and attributes. A constraint is written as a text and appears in a rectangle enclosed with braces with a folded upper-right corner. You can use a constraint on a single element, such as a class or an association path; the constraint text may be positioned near the name of the element and placed in a note symbol.

2. A state shows a graphical depiction of the status of an object. It usually reflects a specific set(s) of its attributes and their relationship. A statechart transition consists of a single arrow between a source and a destination and shows a relationship between two states. Transitions denote the reaction to a message in a given state.
   Statecharts are used to describe the run-time behaviour of objects of a class. Each statechart is included at a class level which describes all behavioural features of the objects in that class. Some classes do not use state charts because their objects are uni-state and always behave similarly.

☞ Check Your Progress - 3

1. The difference between transient objects and persistent objects are as follows:
   Persistent object is an instance of a class in the domain model that represents its values in databases, whereas a transient object is an instance of a class in the domain model which is created in memory. The garbage collector will remove the transient objects if they don't have any use and reference in the database. The persistent objects are saved in the databases. For example, if you acquire an entity from a database, then that entity is persistent. When you create a new entity, it is transient until it is stored in a database. The persistence in Object Oriented Databases is handled by creating Persistent Objects and Transient Objects.
2. Persistent data is data which has a longer lifetime than the program that created it. Enabling data to be stored on a permanent storage medium provides persistency. The most common techniques used are storing data in the form of files or making use of a back-end database system.
3. The Persistence data is accessible after closing the application. This type of data must be saved into a database or internal or external memory.
   Non- persistence data is not accessible after closing the application. It means that it is volatile data available during the application execution only.
4. Serialization provides an appropriate and straightforward approach of making data persistent. Serialization transforms an object into byte streams to store the object in memory or a database. It is most suitable when the amount of data included is relatively small. If more amounts of data are to be stored, serialization may no longer be an appropriate approach for storage.

13.8 REFERENCES/FURTHER READINGS

- Kenneth Barclay and John Savage, " Object-Oriented Design with UML and Java", 2003.
- James Rambaugh, Michael Blaha, William Premerlam, Frederick Eddy, and William Corensen, "Object Oriented Modelling and Design" PHI, New Delhi, 2004.
- https://www.uml-diagrams.org/communication-diagrams.html
- https://www.uml-diagrams.org/communication-diagrams-examples.html
- https://www.uml-diagrams.org/constraint.html